1. 객체 선언 **(Object Declaration) —** 싱글톤 패턴

- 개념**:** 프로그램 전체에서 단 하나의 인스턴스만 존재해야 하는 객체(싱글톤)를 생성할 때 사용합니다.
- 특징**:** 생성자를 가질 수 없으며, 코드에서 해당 객체에 처음 접근할 때 자동으로 인스턴스가 1회 생성됩니다.
- 주요 용도**:** 앱 전체 공통 설정(Config), 로그 관리자, 데이터베이스 연결 객체 등

```
object AppConfig {
    val appName = "MyApp"
    fun printInfo() = println("앱 이름: $appName")
}
// 호출: AppConfig.printInfo()
```

2. 동반 객체 **(Companion Object) —** 정적**(static)** 멤버 대체

- 개념**:** 자바의 `static` 키워드 대신, 클래스 인스턴스 생성 없이 클래스 이름으로 직접 호출할 프로퍼티나 함수를 만들 때 클래스 내부에 선언합니다.
- 특징**:** 클래스당 1개만 선언할 수 있으며, 자신이 속한 클래스의 `private` 생성자 및 멤버에도 접근할 수 있어 팩토리 메서드(객체 생성 대행 함수) 구현에 유용합니다

```
class User(val name: String) {
    companion object {
        const val MAX_AGE = 100
        fun createGuest() = User("Guest")
    }
}
// 호출: println(User.MAX_AGE) / val guest = User.createGuest()
```

3. 객체 표현식 **(Object Expression) —** 익명 내부 클래스

- 개념**:** 이름 없는 일회용 객체(자바의 익명 내부 클래스)를 즉석에서 생성하여 인터페이스나 추상 클래스를 구현할 때 사용합니다.
- 특징**:** 싱글톤이 아니며, 객체 표현식이 실행될 때마다 매번 새로운 인스턴스가 생성됩니다.

```
interface OnClickListener { fun onClick() }

val listener = object : OnClickListener {
    override fun onClick() = println("버튼 클릭됨")
}
```